API Data Flow Reference (Server)

How to use this document

This document is a living reference updated at the end of each sprint. Each endpoint is documented with four components:

- Request summary – method, auth requirement, body fields, and success response.
- Data flow – a step-by-step table showing exactly which file handles each stage of the request, from route to database.
- Error cases – every failure scenario with its error code and response message.
- Notes – any design decisions or caveats specific to that endpoint.

Architecture pattern followed by every endpoint:

Route → Middleware (protected routes only) → Controller → Service → Database

Token architecture: two-token system. Access token (15m expiry) sent in Authorization header for every protected request. Refresh token (7d expiry) stored in httpOnly cookie, used only to issue new access tokens without requiring re-login.

1. Auth

Covers user registration, login, logout, and token refresh. Register and login are public endpoints. Logout and refresh-token require a valid Bearer access token.

1.1 POST /api/v1/auth/register

Creates a new user account. Accepts name, email, password, and role. Writes two rows atomically – one to the USERS table (credentials) and one to the role-specific table (profile name + userID).

Request summary

Method	POST
Auth required	No – public endpoint
Body fields	name, email, password, role
Valid roles	admin, adopter, staff, vet, volunteer, donor
Success	201 – { userID, userEmail, role }

Data flow

File	What happens here
auth.routes.js	Receives the POST request. No middleware – public route. • Maps POST /register to the register controller function
auth.controller.js	Validates all 4 fields are present and role is valid, then calls the service.

File	What happens here
	• Destructures { name, email, password, role } from req.body • Returns VALIDATION_ERROR (422) if any field is missing • Returns VALIDATION_ERROR (422) if role is not one of the 6 valid values • Calls authService.register() inside try/catch • On success: calls successResponse with 201 and the created user object • On failure: calls next(err) – triggers errorHandler middleware
auth.service.js	Checks for duplicate email, hashes password, writes two rows atomically via a Prisma transaction. • Queries USERS table with findUnique – throws CONFLICT (409) if email already exists • Hashes the password with bcrypt (10 salt rounds) • Opens a Prisma $transaction: inserts into USERS, then inserts into the role table using the returned userID • If either insert fails, the entire transaction rolls back – no orphaned records • Strips userPassword and refreshToken from the returned object before passing back to controller
USERS table	Stores authentication credentials shared across all roles. • Columns written: userEmail, userPassword (hashed), role
Role table	Stores name and userID for the specific role (e.g. Adopter, Staff). • Columns written: userID (FK → USERS), [role]Name (e.g. adopterName)

Error cases

Scenario	Error Code	Response Message
Missing any of the 4 fields	**VALIDATION_ERROR**	name, email, password, and role are required
Role not in the allowed list	**VALIDATION_ERROR**	role must be one of: admin, adopter, staff, vet, volunteer, donor
Email already registered	**CONFLICT**	A user with this email is already registered
DB insert fails	**INTERNAL_SERVER_ERROR**	Internal server error

1.2 POST /api/v1/auth/login

Authenticates an existing user. Verifies credentials against the USERS table and issues two tokens: a short-lived access token (15m) in the response body and a long-lived refresh token (7d) in an httpOnly cookie.

Request summary

Method	POST
Auth required	No – public endpoint
Body fields	email, password
Success	200 – { token (access token, 15m), user: { userID, userEmail, role } } + refreshToken httpOnly cookie (7d)

Data flow

File	What happens here
auth.routes.js	Receives the POST request. No middleware – public route. • Maps POST /login to the login controller function
auth.controller.js	Validates email and password, calls the service, signs both tokens, and sets the cookie. • Destructures { email, password } from req.body • Returns VALIDATION_ERROR (422) if either field is missing • Calls authService.login() inside try/catch • Signs access token: { userID, role }, 15m expiry, using JWT_SECRET • Signs refresh token: { userID }, 7d expiry, using JWT_SECRET • Hashes the refresh token with bcrypt and calls authService.storeRefreshToken(userID, hashedRT) • Sets refreshToken as httpOnly cookie with sameSite: strict • Returns 200 with { token: accessToken, user } via successResponse
auth.service.js (login)	Verifies credentials and returns the safe user object. • Queries USERS table with findUnique({ where: { userEmail: email } }) • Throws NOT_FOUND (404) if no record returned • Calls bcrypt.compare(password, user.userPassword) • Throws UNAUTHORIZED (401) if password does not match • Strips userPassword and refreshToken from returned object
auth.service.js (storeRefreshToken)	Stores the hashed refresh token against the user record. • Calls prisma.users.update({ where: { userID }, data: { refreshToken: hashedRT } })
USERS table	Queried for credentials; refreshToken column updated with new hash. • Columns read: userEmail, userPassword • Columns written: refreshToken (hashed)

Error cases

Scenario	Error Code	Response Message
Missing email or password	VALIDATION_ERROR	email and password are required
Email not found in USERS	NOT_FOUND	No account found with this email
Password does not match hash	UNAUTHORIZED	Incorrect password

1.3 POST /api/v1/auth/logout

Request summary

Method	POST
Auth required	Yes – Bearer access token in Authorization header
Body fields	None
Success	200 – { message: "Logged out successfully" }

Data flow

File	What happens here
auth.routes.js	Registers authenticate middleware before the controller – protected route. • Maps POST /logout to: authenticate middleware → logout controller • If authenticate fails the request never reaches the controller
authenticate.js	Validates the access token from the Authorization header. • Extracts Bearer token from Authorization header – returns UNAUTHORIZED if missing • Verifies JWT signature using JWT_SECRET – returns UNAUTHORIZED if invalid or expired • Attaches decoded { userID, role } to req.user and calls next()
auth.controller.js	Calls the service with userID and clears the cookie. • Uses req.user.userID set by authenticate middleware • Calls authService.logout(userID) inside try/catch • Calls res.clearCookie('refreshToken') to remove the httpOnly cookie • Returns 200 with 'Logged out successfully'
auth.service.js	Nullifies the stored refresh token in the DB. • Calls prisma.users.update({ where: { userID }, data: { refreshToken: null } }) • Any future attempt to use the old refresh token will fail the bcrypt comparison
USERS table	refreshToken column set to null. • Columns written: refreshToken = null

Error cases

Scenario	Error Code	Response Message
No Authorization header present	UNAUTHORIZED	Authentication failed – no token provided
Access token invalid or expired	UNAUTHORIZED	Authentication failed – invalid token

1.4 POST /api/v1/auth/refresh-token

Issues a new access token and rotates the refresh token. Called by the frontend when the access token expires (401 response received). The old refresh token is invalidated on every rotation – if the same refresh token is used twice it will be rejected.

Request summary

Method	POST
Auth required	Yes – Bearer access token in Authorization header (can be expired)
Cookie required	refreshToken httpOnly cookie
Body fields	None
Success	200 – { token (new access token, 15m), user: { userID, role } } + new refreshToken httpOnly cookie (7d)

Data flow

File	What happens here
auth.routes.js	Registers authenticate middleware before the controller. • Maps POST /refresh-token to: authenticate middleware → refreshToken controller
authenticate.js	Validates the access token from the Authorization header. • Note: the access token may be expired – authenticate uses jwt.verify() which will reject expired tokens • If the access token is expired the frontend should still send it so the controller can decode userID • Attaches decoded { userID, role } to req.user if valid
auth.controller.js	Reads the old refresh token from the cookie, signs the new tokens, calls the service to rotate. • Reads rawOldRT from req.cookies.refreshToken – returns UNAUTHORIZED if missing • Decodes the access token with jwt.decode() to get userID (no verification – just payload extraction) • Signs new refresh token: { userID }, 7d expiry • Calls authService.refreshToken(userID, rawOldRT, newRefreshToken) inside try/catch • On success: signs new access token { userID, role } with 15m expiry • Sets new refreshToken as httpOnly cookie • Returns 200 with new access token and { userID, role }

File	What happens here
auth.service.js	Verifies the old refresh token against the DB hash, stores the new hash. • Queries USERS table by userID – throws NOT_FOUND if user missing • Throws UNAUTHORIZED if user.refreshToken is null (already logged out) • Calls bcrypt.compare(rawOldRT, user.refreshToken) – throws UNAUTHORIZED if no match • Hashes the new refresh token: bcrypt.hash(rawNewRT, 10) • Updates USERS.refreshToken with the new hash • Returns safe user object (userPassword and refreshToken stripped)
USERS table	refreshToken column read for verification and updated with new hash. • Columns read: refreshToken (hash for bcrypt comparison), role • Columns written: refreshToken (new hash)

Error cases

Scenario	Error Code	Response Message
No refreshToken cookie present	**UNAUTHORIZED**	No refresh token provided
No Authorization header present	**UNAUTHORIZED**	Authentication failed – no token provided
Refresh token not in DB (logged out)	**UNAUTHORIZED**	Session expired, please log in again
Refresh token does not match DB hash	**UNAUTHORIZED**	Invalid refresh token
User not found in DB	**NOT_FOUND**	User not found

Design note

Refresh token rotation means each refresh token can only be used once. After a successful refresh the old token is invalidated in the DB. If an attacker steals a refresh token and tries to use it after the legitimate user has already rotated it, the bcrypt comparison will fail and the request will be rejected.

Supporting files

These files are shared across all endpoints and not specific to any single flow.

File	Purpose
config/prisma.js	Singleton Prisma client – the single gateway to the database. Imported by all service files.
utils/response.js	successResponse() and errorResponse() – standardise all API response shapes.

File	Purpose
utils/errors.js	ERROR_CODES map – defines valid error codes and their HTTP status codes (400, 401, 403, 404, 409, 422, 500).
middleware/errorHandler.js	Global error handler – registered last in index.js. Receives errors via next(err) and formats them using errorResponse().
middleware/authenticate.js	JWT verification middleware – used on all protected routes. Validates the Bearer access token and attaches req.user.
middleware/authorizeRoles.js	Role-based access control – used on routes that require a specific role. Not yet applied in Sprint 1 auth endpoints.
index.js	Express app entry point. Registers middleware, mounts routers.